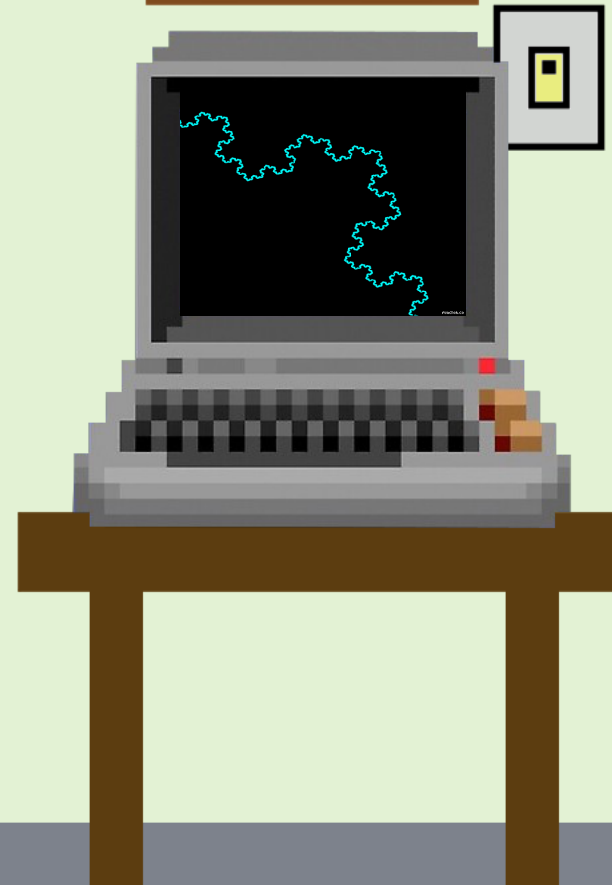


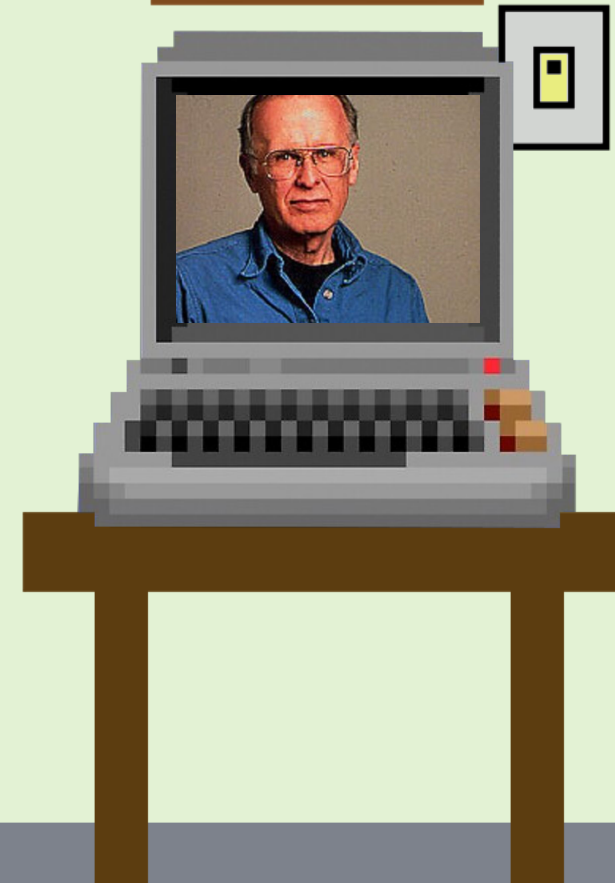
Paradigma Funcional

Programação Funcional



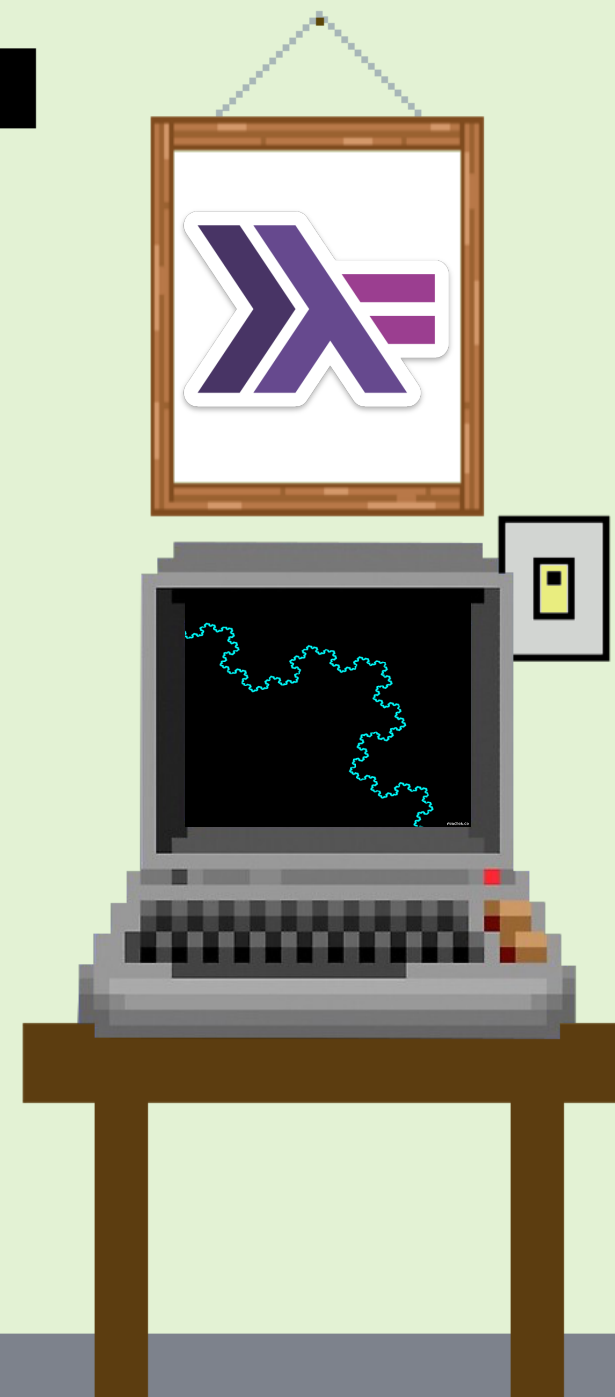
Paradigma Funcional

- Baseado em funções matemáticas
- John Backus (1978)
 - Prêmio Turing pelo desenvolvimento do Fortran
 - Programas funcionais são mais legíveis, mais confiáveis e mais propensos a serem corretos
 - Além disso, são mais fáceis de entender porque o significado das expressões independe do contexto



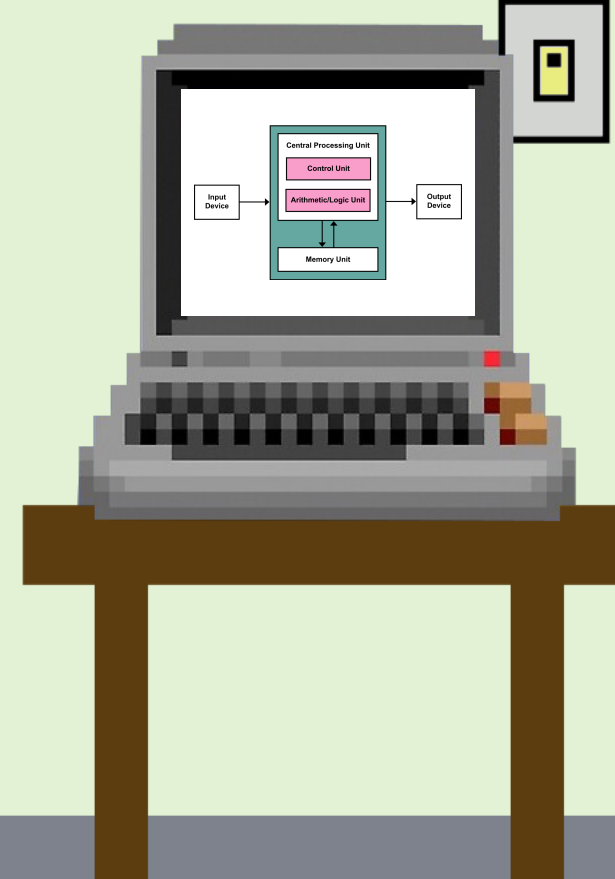
Paradigma Funcional

- Alta expressividade
- Alto poder de abstração
- Polimorfismo paramétrico
- Funções de alta ordem (ordem superior)
- Avaliação parcial
- Avaliação preguiçosa (lazy evaluation)
- Rápida codificação
- Menor número de linhas de código



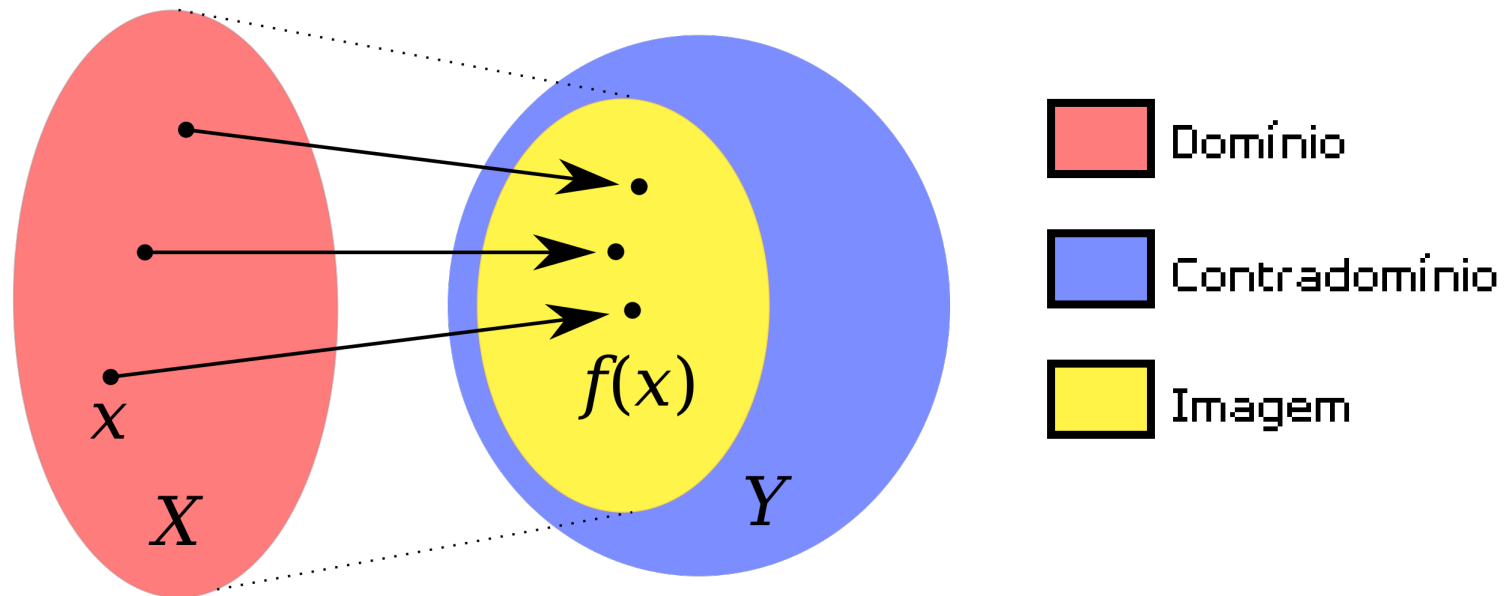
Quebra de Paradigmas

- Arquitetura Von Neumann
- Programas imperativos tem um estado que muda ao longo da execução
 - Variáveis do programa
- Devemos entender os usos de suas variáveis e compreender as mudanças de estado
- Execução por meio da avaliação de expressões e atribuições que alteram o estado do programa
 - Na matemática não existem estados em funções
 - Programas escritos em uma linguagem funcional pura não têm variáveis nem estado

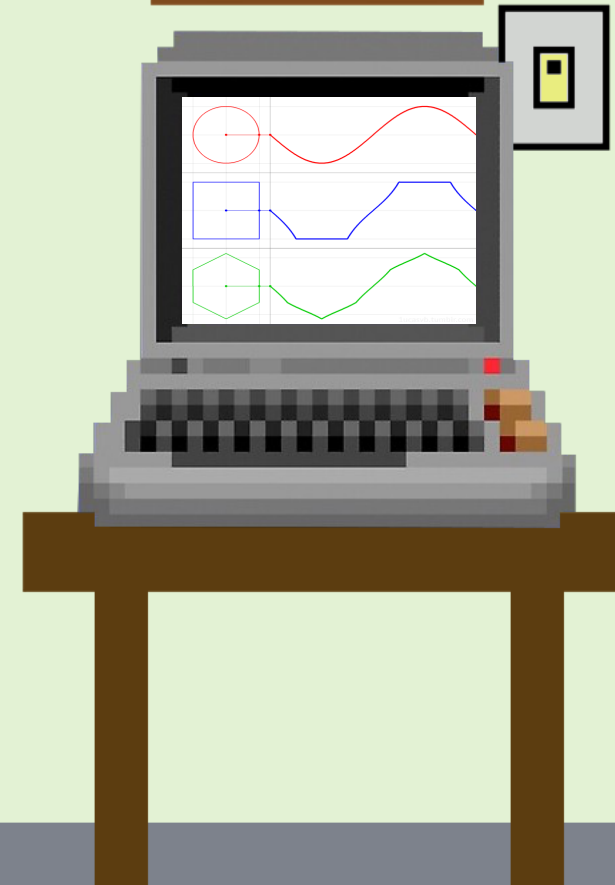


Funções Matemáticas

- Mapeamento de membros do conjunto domínio para o conjunto imagem

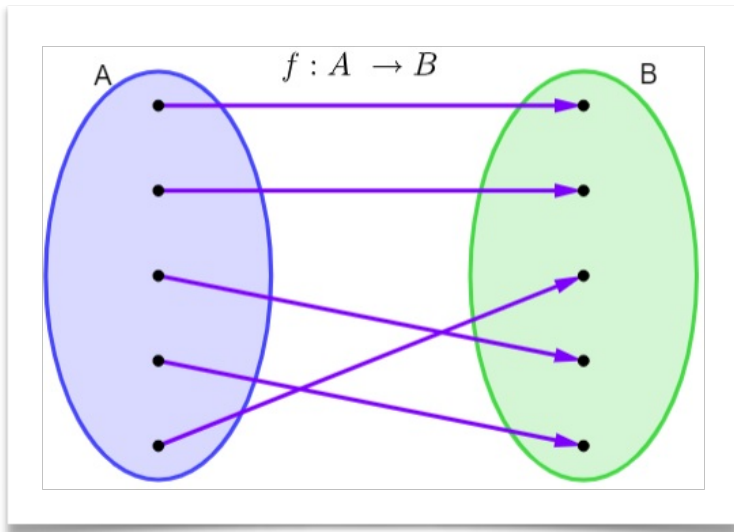


$$f : X \rightarrow Y$$

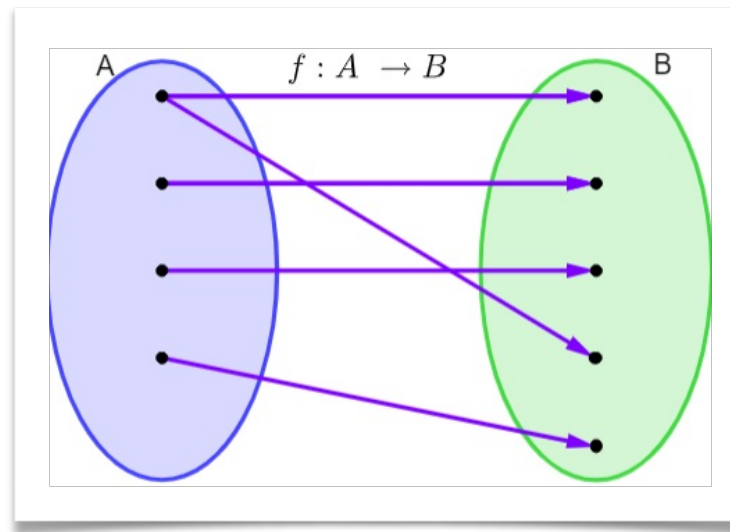


Funções Matemáticas

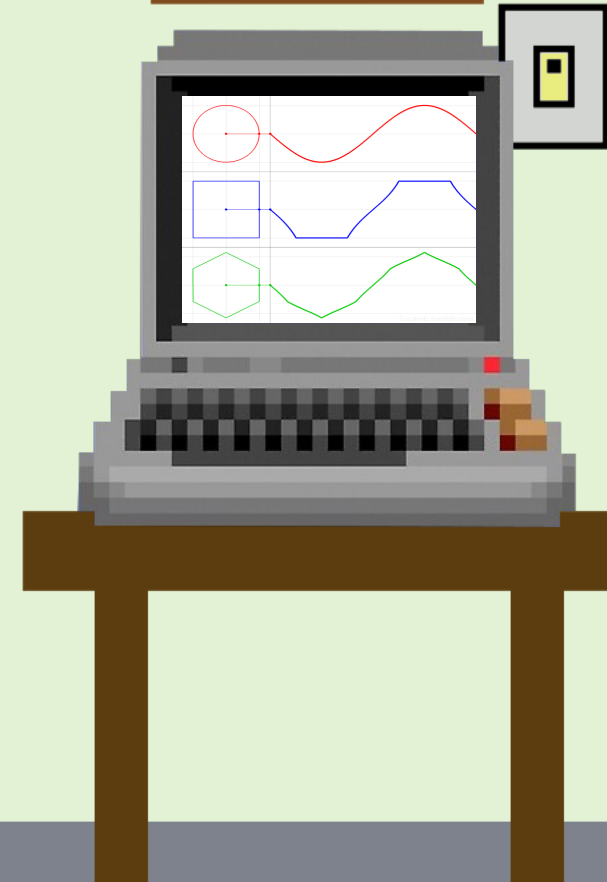
- A definição de uma função especifica o mapeamento entre os membros dos conjuntos domínio e imagem



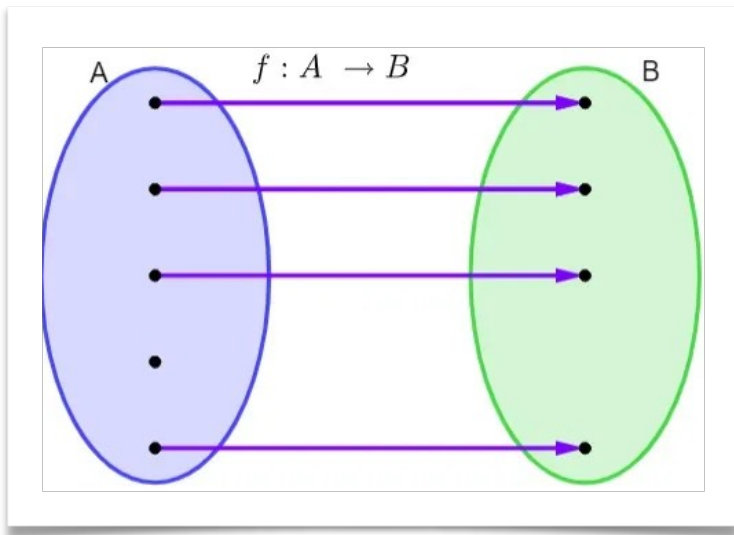
É função



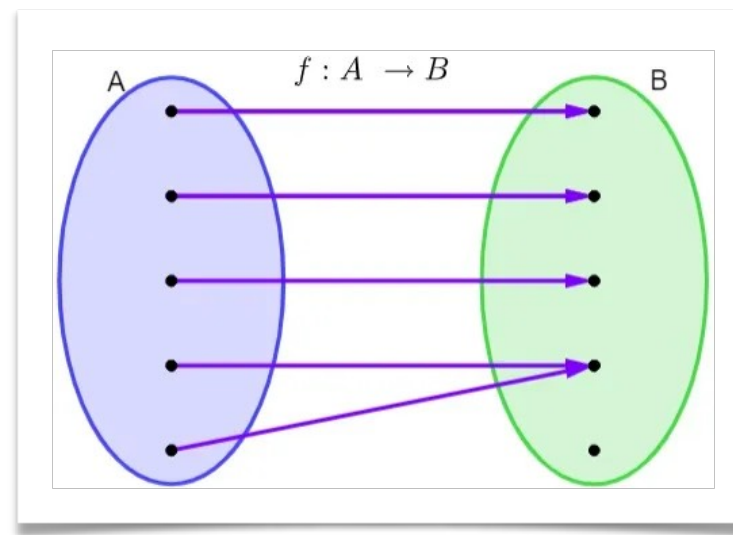
Não é função



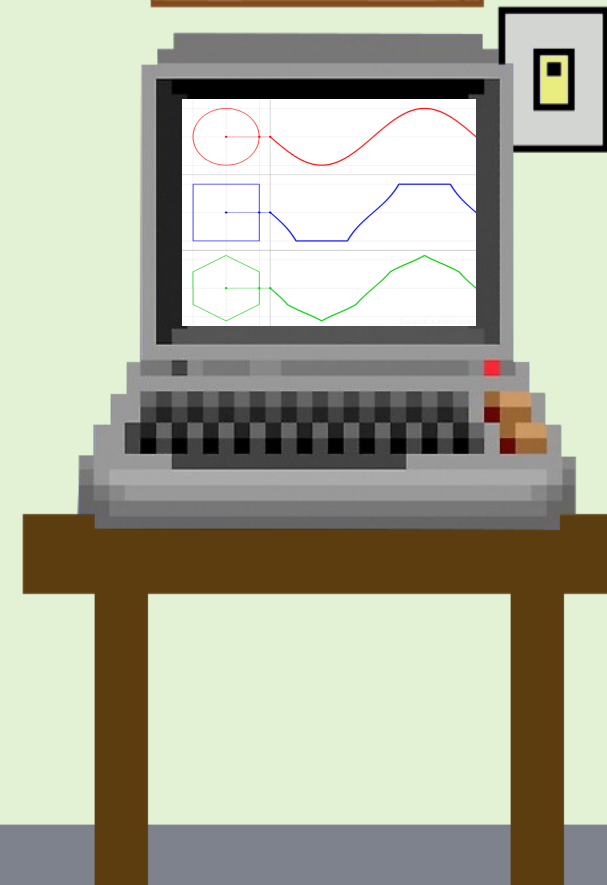
Funções Matemáticas



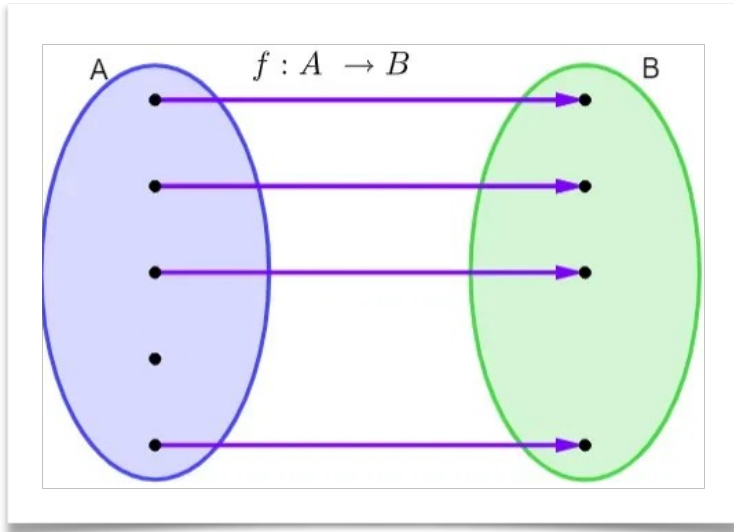
?



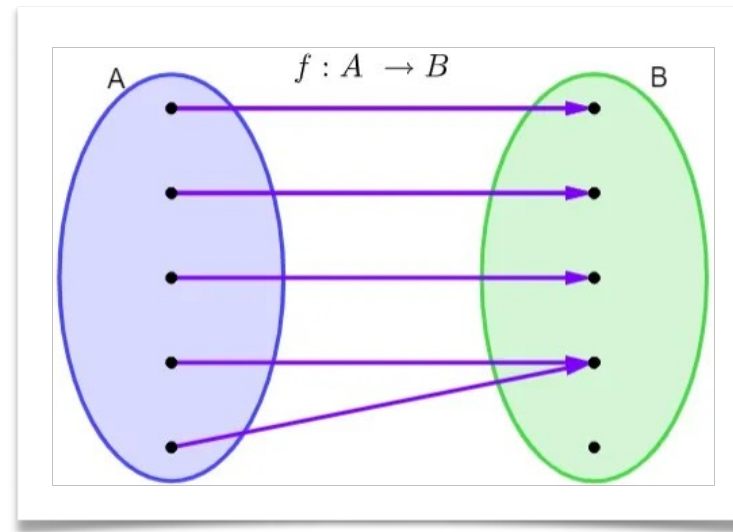
?



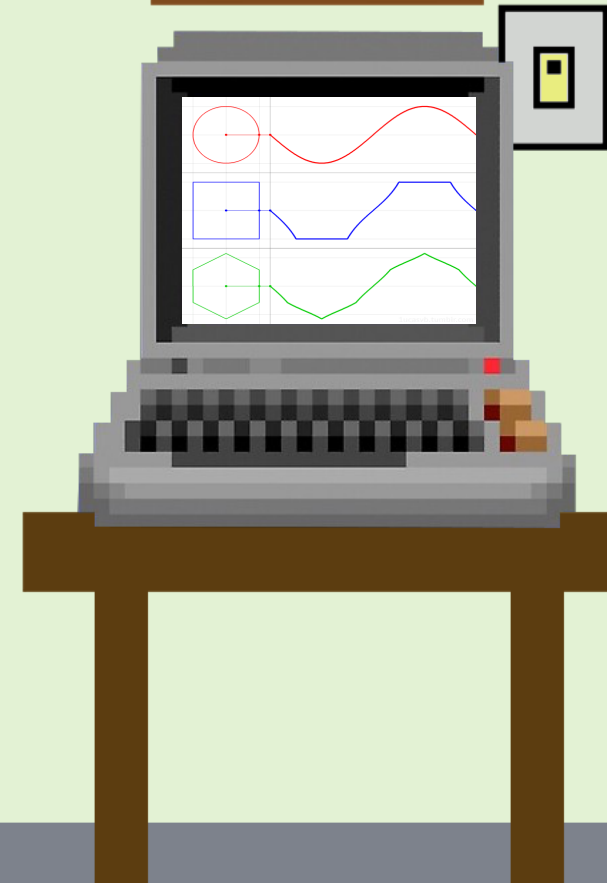
Funções Matemáticas



Não é Função

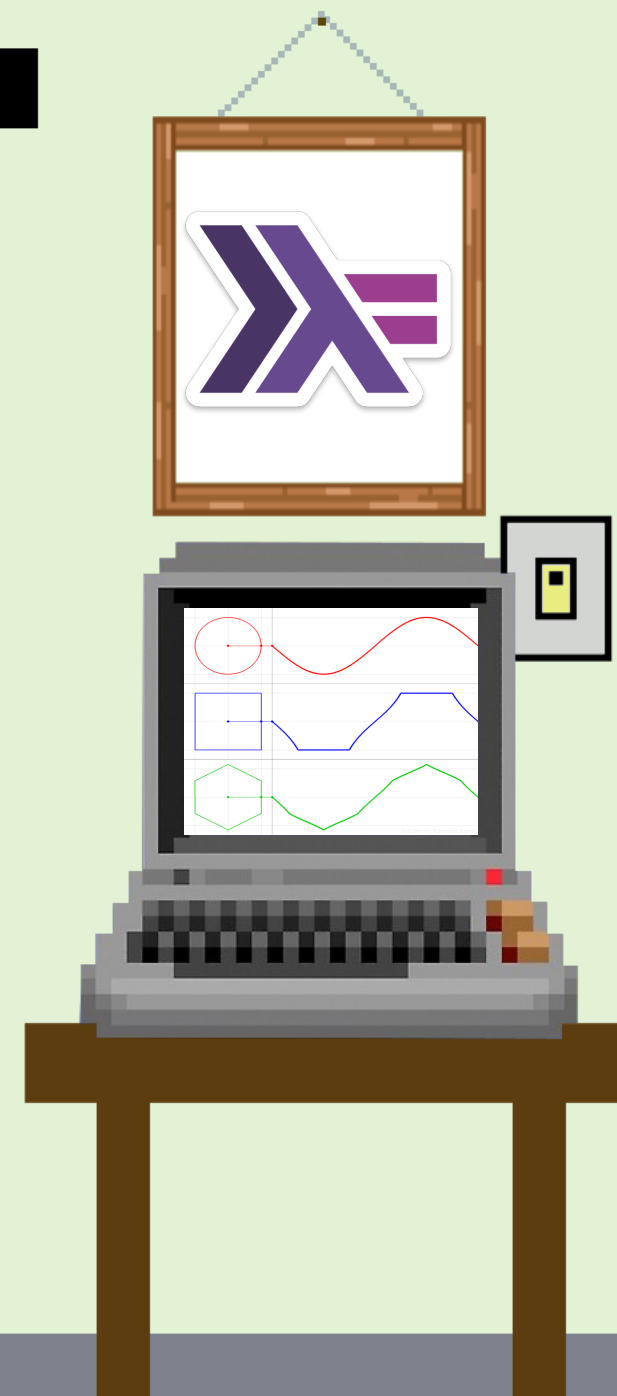


É Função



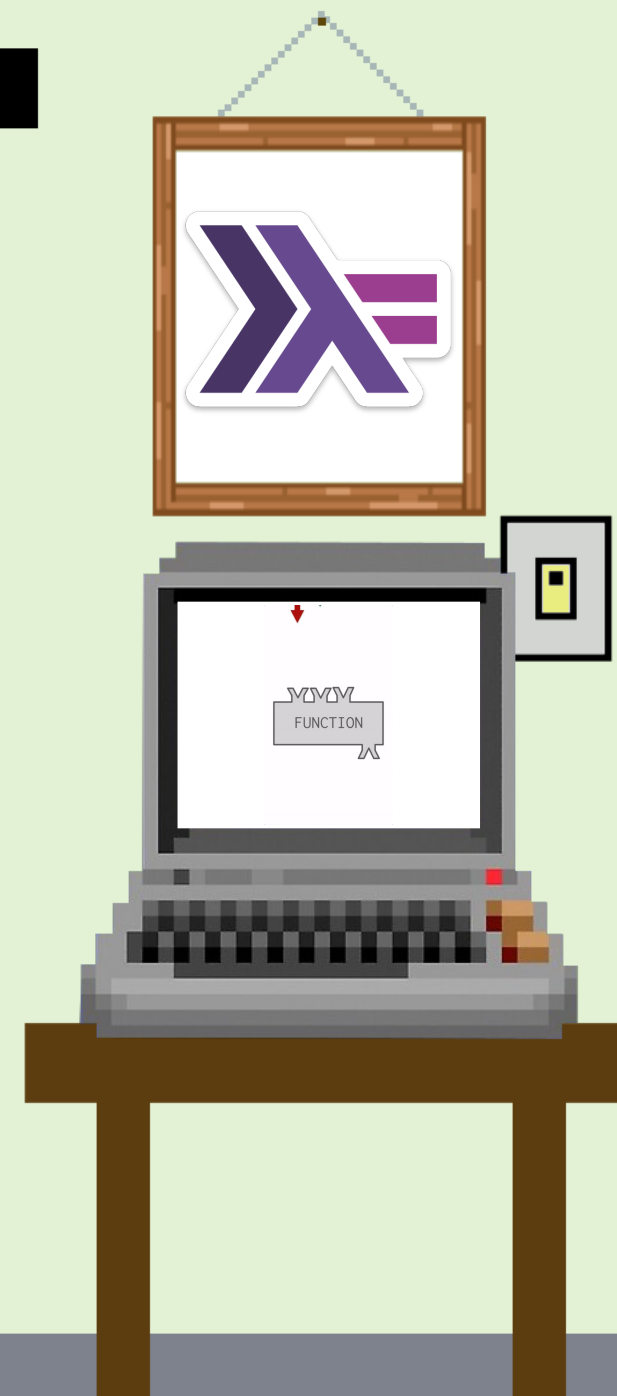
Propriedades Importantes das Funções

- Sempre mapeiam um elemento específico do domínio no mesmo elemento da imagem
- Não dependem de valores externos
- Não possuem efeitos colaterais
- Ordem de avaliação das expressões controlada por recursão e por expressões condicionais
 - Não há sequência ou repetição iterativa



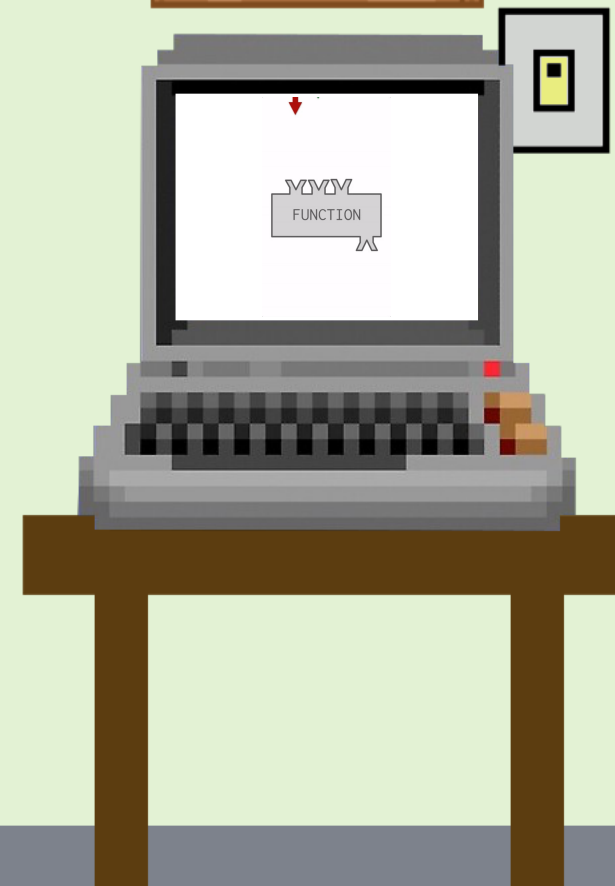
Linguagens Funcionais

- Visa emular funções matemáticas ao máximo possível
- Resolução de problemas fundamentalmente diferente das linguagens imperativas
- Programação de nível mais alto
 - Não requer atenção com os valores das variáveis e o estado do programa
- Os programas são definições e aplicações de funções
- A execução de uma função sempre produz o mesmo resultado quando fornecidos os mesmos parâmetros
 - Transparência referencial



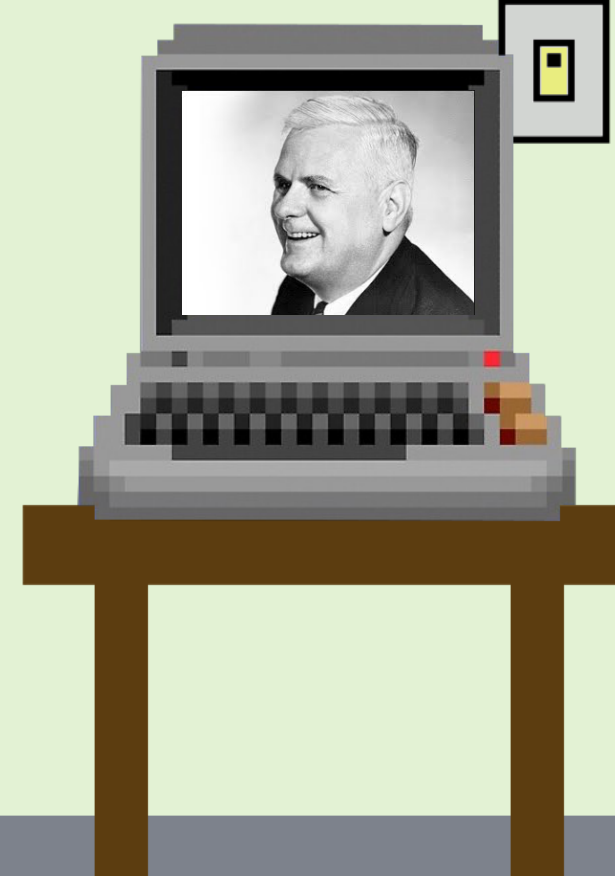
Linguagens Puramente Funcionais

- Não usam variáveis nem sentenças de atribuição
- O programador não se preocupa com as células de memória ou estado do programa
- Não existem construções de iteração
- As repetições devem ser especificadas com recursão
- Não existem efeitos colaterais



Histórico

- λ -calculus
 - Noção de função computável (Alonzo Church - 1930)
- Lisp
 - Primeira linguagem funcional (John McCarthy - 1959)
 - Iniciou pura mas adquiriu recursos imperativos
- Dialetos de Lisp
 - Scheme (1975)
 - Common Lisp (anos 80)
- Linguagens funcionais tipadas
 - ML, Haskell, OCaml, F#, etc.
 - Expansão em outras áreas



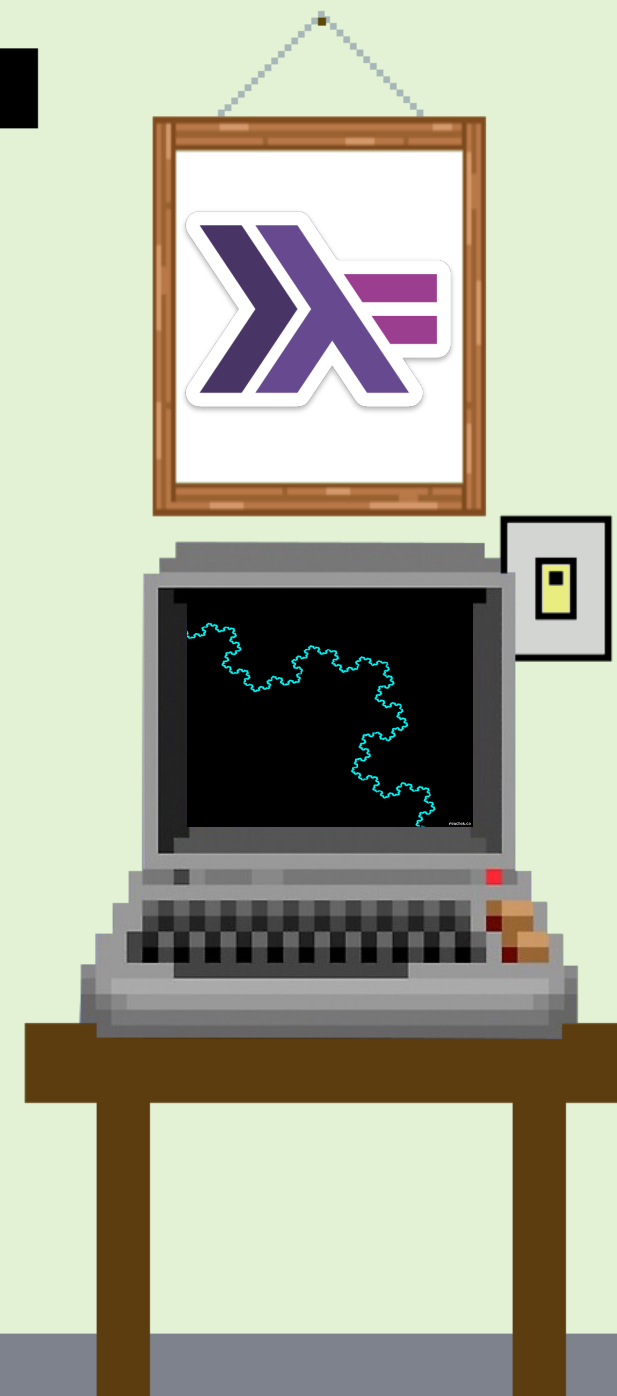
Histórico

- Erlang foi banida pela Ericsson em 1998
 - Distribuída de forma open-source
 - Voltou a ser utilizada pela Ericsson em 2004
- A Microsoft lança o F# (2005)
 - Linguagem primariamente funcional para .NET
- Linguagens OO como Java e C# adotam conceitos típicos de linguagens funcionais (anos 2010)
- O Twitter reescreve seu back-end em Scala
 - Linguagem funcional e OO para a plataforma Java
- Google publica artigos mostrando o uso do paradigma funcional para gerenciar dados



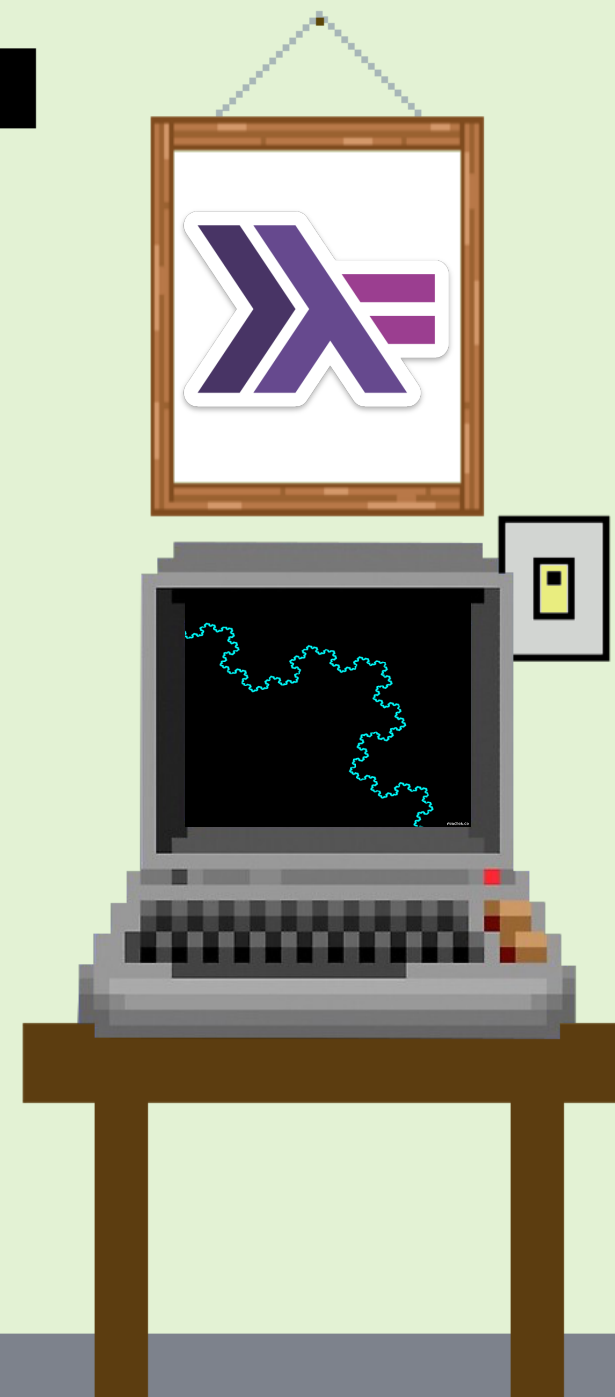
Popularização de Linguagens Funcionais

- Linguagens Funcionais ficaram isoladas muito tempo
 - Linguagens devem disponibilizar formas para integrar seus módulos a diferentes linguagens
- Grande parte do poder de uma linguagem está nas bibliotecas disponibilizadas com ela
 - Ruby obteve grande sucesso devido ao Rails
- Risco em apostar em linguagens pouco populares



Popularização de Linguagens Funcionais

- Havia poucas ferramentas para desenvolvimento
 - Debuggers, profilers e IDEs
- Programadores imperativos resistentes em migrar
- O desempenho dos programas era pior
 - Arquitetura não otimizada para o paradigma
- Dificuldade na portabilidade e instalação de compiladores e interpretadores
 - Ex: Instalar o GHC era um desafio



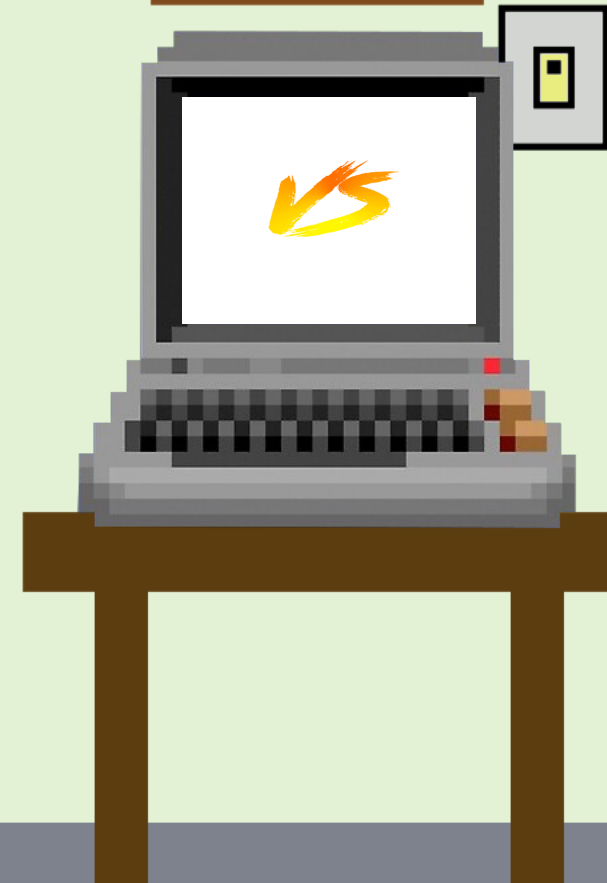
Funcional vs Imperativo

```
total = 0;  
for (int i=1; i<=10; ++i)  
    total = total + i;
```

passo	instrução	i	total
1	total=0	?	0
2	i=1	1	0
3	total=total+i	1	1
4	++i	2	1
5	total=total+i	2	3
6	++i	3	3
7	total=total+i	3	6
8	++i	4	6
9	total=total+i	4	10
:	:	:	:
21	total=total+i	10	55
22	++i	11	55



vs



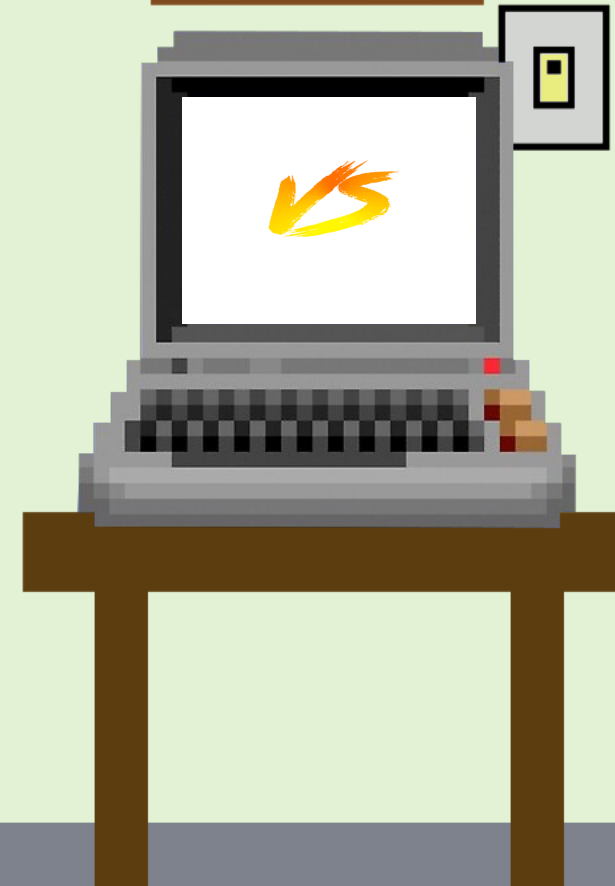
Funcional vs Imperativo

sum [1..10]

```
sum [1..10] =  
= sum [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] =  
= 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 =  
= 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 =  
= 7 + 5 + 6 + 7 + 8 + 9 + 10 =  
= 12 + 6 + 7 + 8 + 9 + 10 =  
:  
= 55
```



vs



Quicksort (Java)

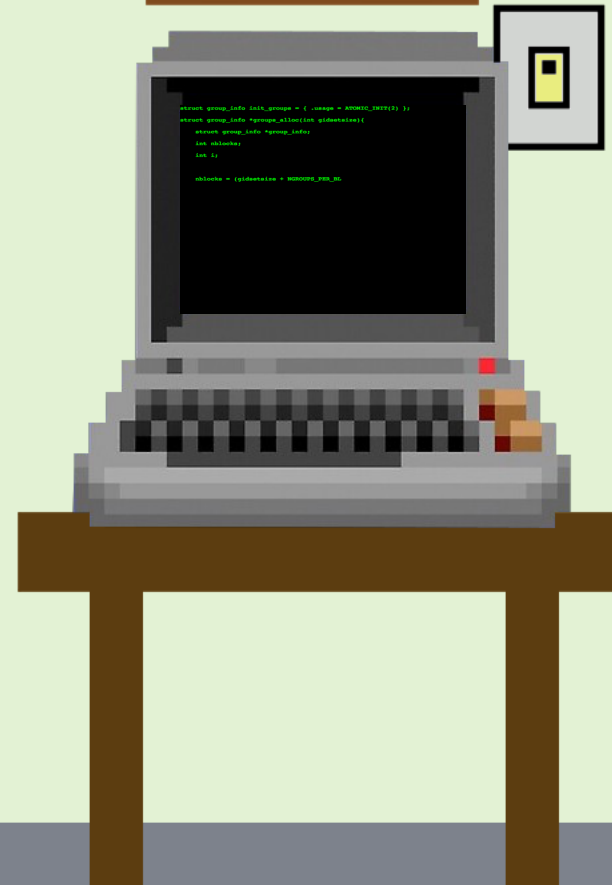
```
void quickSort(int arr[], int begin, int end) {  
    if (begin < end) {  
        int partIdx = partition(arr, begin, end);  
        quickSort(arr, begin, partIdx-1);  
        quickSort(arr, partIdx+1, end);  
    }  
}
```

```
int partition(int arr[], int begin, int end) {  
    int pivot = arr[end];  
    int i = (begin-1);  
    for (int j = begin; j < end; j++) {  
        if (arr[j] <= pivot) {  
            i++;  
            int swapTemp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = swapTemp;  
        }  
    }  
    int swapTemp = arr[i+1];  
    arr[i+1] = arr[end];  
    arr[end] = swapTemp;  
    return i+1;  
}
```



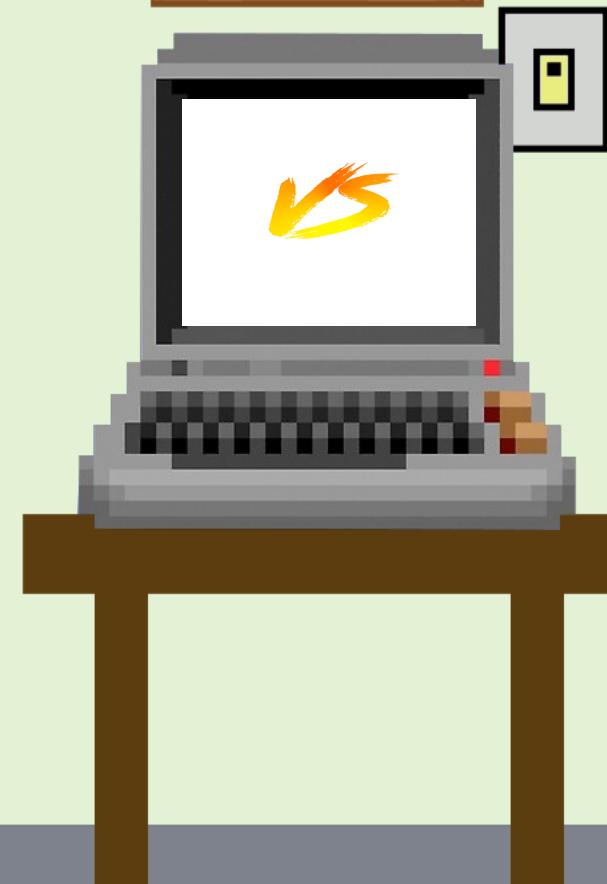
Quicksort (Haskell)

```
qsort [] = []  
qsort (x:xs) = qsort xs1 ++ [x] ++ qsort xs2  
    where xs1 = [x' | x'<-xs, x'<=x]  
          xs2 = [x' | x'<-xs, x'>x]
```



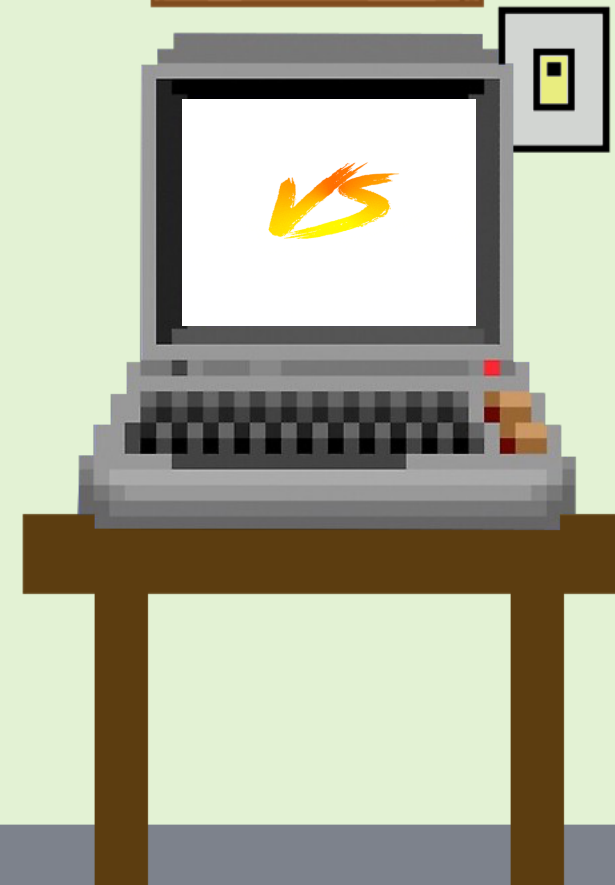
Funcional vs Imperativo

- Nome e Valores
 - Imperativas - O mesmo nome pode ter diferentes valores ao longo da execução
 - Funcionais - Nomes associados a um valor único
- Ordem de Execução
 - Imperativas - Determina o resultado
 - Funcionais - Resultado independente da ordem
- Repetições
 - Imperativas - Comandos de controle
 - Funcionais - Realizado por chamadas recursivas



Funcional vs Imperativo

- Estruturas de Dados
 - Imperativas - Valores internos podem ser alterados
 - Funcionais - Subestruturas imutáveis
- Funções
 - Imperativas - Raramente podem ser tratadas como valor ou armazenadas em variáveis
 - Funcionais - Elementos de alta ordem
- Avaliações de Expressões
 - Imperativas - Avaliadas antes da invocação (eager)
 - Funcionais - Avaliadas quando necessário (lazy)



Paradigma Funcional em Linguagens Imperativas

- Algumas linguagens imperativas fornecem suporte parcial para programação funcional
 - Muitas não suportam funções de ordem superior
 - Ex: funções anônimas ou expressões lambda em JavaScript, Python, Ruby, Java e C#
- Existem linguagens multiparadigmas com foco no aspecto funcional
 - F#, Elixir, OCaml, Kotlin, Scala, etc.

